1. Podstawy skryptów i struktura

- **Shebang (`#!`):** Pierwsza linia skryptu (np. `#!/bin/bash`) wskazuje interpreter, który ma wykonać kod, niezależnie od aktualnie używanej powłoki.
- **Komentarze:** Znak # oznacza początek komentarza – wszystko, co się po nim znajduje w danej linii, jest pomijane.
- **Uruchamianie:** Aby plik stał się skryptem wykonywalnym, należy nadać mu odpowiednie uprawnienia poleceniem `chmod +x nazwa_skryptu`, a następnie uruchomić go np. za pomocą `./nazwa_skryptu`.

2. Polecenie `echo`

Służy do wypisywania tekstów na ekranie (standardowe wyjście `stdout`).

- Pozwala na modyfikowanie sposobu wyświetlania za pomocą opcji:
  - `-n` – nie wysyła znaku nowej linii na końcu,
  - `-e` – włącza interpretację znaków specjalnych (np. `\n` – nowa linia, `\t` – tabulacja, `\a` – dzwonek alarmowy).
- Można je przekierowywać do plików za pomocą operatorów `>` (nadpisanie pliku) oraz `>>` (dopisanie na końcu).

3. Cytowanie i zmienne maskujące

- **Cudzysłów (`" "`):** Pozwala na podstawianie wartości zmiennych (np. `$zmienna`), ale zachowuje specjalne znaczenie znaków takich jak `$` czy `` ` `` (odwrotny apostrof).
- **Apostrof (`' '`):** Traktuje wszystko dosłownie jako zwykły tekst – wyłącza interpretację wszelkich znaków specjalnych.
- **Odwrotny apostrof (`` ` ``) lub `$()`:** Służy do zacytowania polecenia i podstawienia jego wyniku pod zmienną (np. `x=$(pwd)`).
- **Backslash (`\`):** Znak maskujący, który wyłącza specjalne znaczenie następującego po nim znaku (np. `\$` wyświetli zwykły znak dolara zamiast odwoływać się do zmiennej).

4. Zmienne

- **Programowe (użytkownika):** Tworzone przez użytkownika (np. `x="napis"`). **Ważne:** po obu stronach znaku `=` nie może być spacji!
- **Specjalne:** Automatyczne zmienne powłoki przechowujące m.in.:
  - `$0` – nazwę bieżącego skryptu,
  - `$1` do `$9` – parametry przekazane do skryptu,
  - `$@` – wszystkie parametry przekazane do skryptu,
  - `$?` – kod powrotu ostatnio wykonanego polecenia,

    - `$$` – PID (identyfikator) bieżącego procesu.
- **Środowiskowe:** Definiują środowisko systemu. Dzielą się na *lokalne* (widoczne tylko w bieżącym shellu) oraz *globalne* (dostępne w podshellach dzięki poleceniu `export`).

5. Zmienne tablicowe

Bash obsługuje tablice jednowymiarowe indeksowane od zera:

- **Deklaracja:** `tablica=(el1 el2 el3)`
- **Odwołanie:** `${tablica[0]}` (pierwszy element) lub `${tablica[*]}` / `${tablica[@]}` (wszystkie elementy).
- **Długość:** `${#tablica[@]}` zwraca liczbę elementów, a `${#tablica[0]}` liczbę znaków w pierwszym elemencie.
- **Modyfikacja i usuwanie:** Dodawanie odbywa się poprzez przypisanie wartości pod indeks (np. `tablica[3]=el4`), a do usuwania elementów lub całej tablicy służy polecenie `unset`.

6. Strumienie danych

Procesy w Linuksie operują na trzech standardowych strumieniach, identyfikowanych przez deskryptory:

- `0` – **stdin** (standardowe wejście, domyślnie klawiatura),
- `1` – **stdout** (standardowe wyjście, domyślnie ekran),
- `2` – **stderr** (standardowe wyjście błędów).
- Strumienie można przekierowywać za pomocą operatorów takich jak `<` (wejście z pliku), `>` (wyjście do pliku), `>>` (dopisanie do pliku) oraz `2>` (kierowanie błędów do pliku).

7. Instrukcje warunkowe

- **`if` / `elif` / `else`:** Służą do podejmowania decyzji na podstawie sprawdzanych warunków. Warunki umieszcza się w nawiasach kwadratowych `[ ... ]` (dla testów tekstowych i podstawowych) lub podwójnych nawiasach okrągłych `(( ... ))` dla wyrażeń arytmetycznych. Pomiędzy nawiasami a treścią **muszą znajdować się spacje**.
- **Polecenie `test`:** Zwraca status `0` (prawda), jeśli warunek jest spełniony, lub `1` (fałsz), gdy nie jest. Posiada szereg operatorów do sprawdzania plików (np. `-e` czy plik istnieje, `-d` czy jest katalogiem) oraz relacji liczbowych (`-gt`, `-lt`, itp.).
- **`case`:** Instrukcja wielokrotnego wyboru, pozwalająca dopasować wartość zmiennej do określonych wzorców (z opcją domyślną `*)`).

8. Pętle i sterowanie przepływem

- **`for`:** Iteruje po elementach listy lub plikach (np. `for x in lista; do ... done`).
- **`while`:** Wykonuje się tak długo, jak długo sprawdzany warunek jest prawdziwy.

- **`until`:** Wykonuje się dopóki warunek nie stanie się prawdziwy (działa odwrotnie do `while`).
- **`select`:** Generuje interaktywne, ponumerowane menu z listy opcji.
- **Sterowanie pętlą:** `break` przerywa działanie pętli, a `continue` pomija bieżącą iterację i przechodzi do kolejnej.

9. Odczyt danych i funkcje

- **`read`:** Służy do pobierania danych wpisywanych przez użytkownika ze standardowego wejścia (np. `read zmienna`). Posiada przydatne opcje, takie jak `-p` (wyświetlenie zachęty) czy `-s` (ukrywanie wpisywanych znaków, np. przy haśle).
- **Funkcje:** Pozwalają na grupowanie bloków kodu w celu ich wielokrotnego użycia. Deklaruje się je jako `nazwa_funkcji() { polecenia; }`, a parametry przekazuje się do nich tak samo jak do zwykłych skryptów (dostępne pod zmiennymi `$1`, `$2` itd.).
- **Wyrażenia arytmetyczne:** Obliczenia na liczbach całkowitych można wykonywać za pomocą konstrukcji `$((wyrażenie))` lub `$[wyrażenie]`, a także za pomocą polecenia `let`.